Restaurant Order Manager

Java Collections Assignment — ArrayList, LinkedList, HashMap & LinkedHashMap

1. Assignment Goal

Build a console-based Restaurant Order Management System. The main goal is to practice choosing and using the appropriate Java collection for different types of data. The project should use ArrayList, LinkedList, HashMap, and LinkedHashMap for clearly different responsibilities.

2. Learning Objectives

- Use ArrayList to store and manage the restaurant menu.
- Use LinkedList to represent the kitchen order queue.
- Use HashMap to find any order quickly using its order ID, regardless of its current status.
- Use LinkedHashMap to keep completed orders in their completion order.
- Work with custom objects inside Java collections.
- Practice adding, removing, searching, and displaying collection data.
- Understand why one collection may be more suitable than another for a specific task.
- Use an enum to represent and control the current status of an order.

3. Required Classes

MenuItem

Attributes: int id, String name, double price, String category

Required behavior: appropriate constructors, getters/setters, and toString().

OrderItem

Attributes: MenuItem item, int quantity

Required methods: getItem(), getQuantity(), calculateSubtotal(). The subtotal is item price × quantity.

Order

Attributes: int orderId, String customerName, ArrayList<OrderItem> items, double total, OrderStatus status

Required methods: addItem(), removeItem(), calculateTotal(), displayOrder(), and a method to update the order status.

Automatic total calculation: The order's total must not rely on the user manually requesting it. Every time addItem() or removeItem() is called, the method must internally call calculateTotal() before returning, so that the "total" field is always up to date. displayOrder() should simply show the already-updated total — it should not be responsible for calculating it.

OrderStatus Enum

Create an enum named OrderStatus to represent the current state of an order. The recommended values are:

- PENDING
- IN_KITCHEN
- COMPLETED
- CANCELLED

An order should normally start as PENDING. When it is added to the kitchen queue, its status becomes IN_KITCHEN. When it is processed, its status becomes COMPLETED. If the order is cancelled, its status becomes CANCELLED. The program should not use plain String values for order status.

Restaurant

This class manages the main collections and contains the main restaurant operations.

Collection	Purpose
ArrayList<MenuItem> menu	Store all menu items.
LinkedList<Order> kitchenQueue	Store orders waiting to be processed.
HashMap<Integer, Order> orders	Store ALL orders ever created: pending, in-kitchen, completed, or cancelled. Orders are never removed from this map.
LinkedHashMap<Integer, Order> completedOrders	Store completed orders while preserving the order in which they were completed.

Important HashMap Rule

The HashMap is the permanent record of all orders created during the program. An order remains in the HashMap even after it is completed or cancelled. Its OrderStatus value tells the restaurant what happened to that order. Do not remove completed or cancelled orders from the HashMap.

4. Required Menu

- 1. Add Menu Item
- 2. Remove Menu Item
- 3. Display Menu
- 4. Search Menu Item
- 5. Create Order
- 6. Add Item to Order
- 7. Remove Item from Order
- 8. Display Order
- 9. Add Order to Kitchen Queue
- 10. Process Next Order
- 11. Search Order
- 12. Check Order Status

- 13. Display Completed Orders
- 14. Cancel Order
- 15. Exit

5. Operation Requirements

Add Menu Item

Read the item's ID, name, price, and category. Add it to the ArrayList menu. Menu item IDs must be unique.

Remove Menu Item

Remove a menu item using its ID. If the item does not exist, display an appropriate message.

Display Menu

Display all menu items stored in the ArrayList.

Search Menu Item

Search for a menu item by ID and display it if found.

Create Order

Read an order ID and customer name. The order ID must be unique. The new order starts with PENDING status and is stored in the HashMap. The order remains in the HashMap for its entire lifecycle.

Add Item to Order

Find the order by ID, find the requested menu item by ID, read the quantity, and add an OrderItem to the order. Items should not be added to orders that are already COMPLETED or CANCELLED.

After the item is added, the order's total must be recalculated automatically (inside addItem()) — the user should never need to trigger the calculation manually.

Remove Item from Order

Find the order and remove the requested item from that order. Items should not be modified after the order is COMPLETED or CANCELLED.

After the item is removed, the order's total must be recalculated automatically (inside removeItem()) so it always reflects the current items in the order.

Display Order

Display the customer, order ID, status, all ordered items, quantities, subtotals, and the final total.

Add Order to Kitchen Queue

Find the order using the HashMap and add it to the LinkedList kitchen queue. Change its status from PENDING to IN_KITCHEN. An order should not be added to the queue more than once.

Process Next Order

Process the first order in the kitchen queue. Use the LinkedList as a queue, change its status to COMPLETED, and add it to the LinkedHashMap of completed orders. Keep the same order in the HashMap.

Before processing, the program must check two conditions: (1) the kitchen queue is not empty, and (2) the order at the front of the queue actually contains at least one OrderItem. An order with an empty items list must NOT be processed — display an appropriate message (e.g. "This order has no items and cannot be processed") and leave it in the kitchen queue instead of moving it to the completed LinkedHashMap.

Search Order

Search for any order by its ID using the HashMap, regardless of whether it is PENDING, IN_KITCHEN, COMPLETED, or CANCELLED.

Check Order Status

Ask the user for an order ID, find the order using the HashMap, and display its current OrderStatus.

Display Completed Orders

Display all completed orders in the same order in which they were completed. The LinkedHashMap should preserve this order.

Cancel Order

This is a new required operation. It lets the restaurant cancel an order that has not been completed yet.

- Ask the user for an order ID.
- Search for the order using the HashMap (orders). If it does not exist, display an appropriate message and stop.
- If the order's status is already COMPLETED or already CANCELLED, reject the cancellation and display an appropriate message — completed or already-cancelled orders cannot be cancelled again.
- If the order's status is IN_KITCHEN, it must also be removed from the LinkedList kitchenQueue, since a cancelled order should no longer wait to be processed.
- If the order's status is PENDING, it simply has not been queued yet, so there is nothing to remove from the kitchen queue.
- In all valid cases, update the order's status to CANCELLED using the OrderStatus enum.
- The order must stay in the HashMap after cancellation (per the Important HashMap Rule) — it must never be deleted, and it must never be added to the completedOrders LinkedHashMap, since it was cancelled, not completed.

6. Order Lifecycle

The order moves through the system while remaining in the HashMap:

- Create Order → PENDING
- Add Order to Kitchen Queue → IN_KITCHEN
- Process Next Order → COMPLETED
- Cancel Order → CANCELLED

The HashMap contains the order in every state. The kitchenQueue only contains orders waiting to be processed (an order leaves the queue either by being processed or by being cancelled), and completedOrders contains completed orders in completion order.

7. Example Data

Menu:

- 1 - Burger - 150 - Main Course
- 2 - Pizza - 200 - Main Course
- 3 - Pasta - 180 - Main Course
- 4 - Cola - 40 - Drinks

Example Order #101:

- Customer: Ahmed
- Status: PENDING
- Burger × 2 = 300
- Cola × 1 = 40
- Total = 340

8. Important Rules

- Do not use arrays as a replacement for the required collections.
- The required collection types must be used for their specified purposes.
- Order IDs and menu item IDs must be unique.
- The HashMap must keep every order created, including completed and cancelled orders.
- An order cannot be processed if the kitchen queue is empty.
- An order cannot be processed if it has no items in it, even if it is at the front of the queue.
- An order should not be processed twice.
- An item cannot be added to an order if it does not exist in the menu.
- Completed or cancelled orders should not be modified.
- An order cannot be cancelled twice, and a completed order cannot be cancelled.
- The order's total must always be recalculated automatically whenever items are added or removed — never left for the user to trigger manually.
- The program should handle invalid IDs and empty collections without crashing.
- Order status must be represented using the OrderStatus enum.
- Keep the program console-based; no database or graphical interface is required.

9. Collection Design Requirement

At the end of the assignment, the student should be able to explain why each collection was selected:

Collection	Why is it used?

ArrayList	Menu storage and indexed/list-style access.
LinkedList	Kitchen queue where the first order is processed first.
HashMap	Permanent record of every order and fast lookup by order ID.
LinkedHashMap	Completed orders with completion/insertion order preserved.

Focus: This assignment is about understanding and applying Java Collections. Students should be prepared to explain not only how their code works, but also why ArrayList, LinkedList, HashMap, and LinkedHashMap were chosen, how the OrderStatus enum controls the order lifecycle, and how Cancel Order and the automatic total recalculation fit into that lifecycle.